Ravi's situation is akin to referring to a Subclass object with a Superclass reference. You get more than you expected. It's a pleasant surprise.

```
SuperclassRef = new SubclassObject
```

Maya's situation is akin to a Superclass object with a Subclass reference. You get less than you expected. It's a tragic mistake.

```
SubclassRef = new SuperclassObject
```

There are four possible conversions that can occur:

1. Refer to a Superclass object with a Superclass reference. This is routine
2. Refer to a Subclass object with a Subclass reference. This is also routine
3. Refer to a Subclass object with a Superclass reference.

This is safe because the Subclass object is an object of its Superclass. Such code can only refer to Superclass methods. You can make an array of Superclass references. Then, you can treat them the same, as if they really were all Superclass objects. As long as you call methods that exist in the Superclass (but were overridden by each of the various Subclasses), then the runtime system calls the correct overridden method for each on. This process is known as Dynamic Binding.

4. Refer to a Superclass object with a Subclass reference.

This is a syntax error! It doesn't make sense. It is only remotely possible if the Subclass is explicitly cast into a Superclass reference, which is a way of telling the compiler you are doing this damn-fool thing with your eyes open. Here's the problem: A Subclass contains more and does more than its Superclass parent. If you use a Subclass reference to point to a Superclass object, you are implying that the object has more data variables available than it really does—and if someone made that assumption—they would be sorely mistaken.

5.5.3 Use Polymorphism

```
class Employee
{
private String ssn;
public Employee()
{
ssn = "";
}
public Employee( String soc_num )
{
ssn = soc_num;
```